When the Scheme Is the Signal: Collection Artifacts and Cross-Corpus Failure in Lexical Phishing-URL Detection

[Author Name(s)]
[Department, Institution, City, Country]
[e-mail address]

***Abstract*—Lexical URL classifiers are routinely reported with 95–99 % accuracy on public phishing benchmarks, yet they fail when applied to differently collected data. Starting from a typical detector — 28 lexical features and a weighted soft-voting ensemble of logistic regression, random forest and XGBoost — we audit its implementation and evaluate it on six public corpora with raw URLs and three live feeds, under random, domain-grouped, external-corpus and leave-one-corpus-out protocols. In the widely used PhiUSIIL corpus every benign URL is an `https://www.` homepage, and across corpora the scheme, the `www.` prefix and URL length change in strength and even in direction of association with phishing; a linear probe identifies the source corpus of a benign URL with 83 % accuracy (chance 17 %). Models trained on PhiUSIIL reach ROC-AUC 0.999 with or without domain-grouped splitting, yet at a threshold set for a 1 % false-positive budget they flag 99–100 % of the benign URLs of four other corpora, their ROC-AUC falls to 0.34–0.78, and adding a slash to an empty URL path flips 100 % of their benign decisions. Removing artifact features and post-hoc calibration do not repair this, and the ensemble is statistically indistinguishable from XGBoost alone. Training on five pooled corpora restores the held-out ROC-AUC of XGBoost to 0.87–0.98 and limits decision flips to at most 15 %, but validation-chosen thresholds still over-flag one unseen corpus and under-flag another. We argue that URL detectors must be trained and evaluated across corpora, on canonicalised URLs and at fixed false-positive budgets, and we release a tested, reproducible pipeline.**

Index Terms*—*phishing detection, malicious URL detection, lexical features, ensemble learning, dataset bias, shortcut learning, cross-dataset generalisation, calibration, explainable AI

I. Introduction

Phishing remains one of the highest-volume and most damaging attack vectors on the web. Most phishing campaigns live for hours rather than days, and the majority of victims are reached before blacklists react [1, 2]. This has motivated *proactive* detectors that judge a URL from its string alone, before the page is fetched. Lexical URL classifiers suit this role well: they need no network access, add negligible latency, and are routinely reported with accuracies between 95 % and 99 % [3, 4, 5, 6, 7, 8].

These figures are difficult to reconcile with what happens when such models leave the dataset they were trained on. Cross-dataset accuracy of URL classifiers has been reported at 51–59 % [9]; detectors "work effectively only when the test sets are split from the same training dataset" [10]; and models trained against homepage-only benign URLs lose 27–39 accuracy points on legitimate login pages [11]. The security machine-learning community has a name for the likely cause — spurious correlations created by sampling bias [12] — and phishing researchers have long warned that benign URLs drawn from popularity lists look structurally different from phishing URLs [13, 14, 15, 16]. What has been missing is a measurement: *which* properties of public URL corpora carry the reported performance, whether they point the same way in every corpus, and whether removing them helps.

This study started from a concrete system. A course-project detector — 28 handcrafted lexical features feeding a weighted soft-voting ensemble of logistic regression, random forest and XGBoost — reported a test accuracy of 96.8 % on a 253,098-URL dataset. When we audited it, its two most influential features (by SHAP) were the URL scheme (`https`) and a subdomain count driven by the `www.` prefix, and it classified `https://google.com/` as malicious with 88 % probability. The rest of this paper shows that this is not an implementation accident but a property of the data the field evaluates on, and quantifies what it does to detection in practice.

We ask three research questions:

- **RQ1.** How much of the in-distribution performance of lexical URL classifiers on public phishing corpora is carried by nameable *collection artifacts* — the scheme, the `www.` prefix, homepage-only benign URLs — and by memorisation of registered domains?
- **RQ2.** Do these artifacts point in the same direction across corpora, and does that explain cross-corpus failure?
- **RQ3.** Does an artifact-aware recipe — URL canonicalisation, domain-grouped model selection and validation-chosen operating thresholds — restore performance on unseen corpora, and do probabilities and thresholds stay reliable under shift?

Our contributions are:

1. An audit and faithful re-execution of a representative lexical detector, exposing an evaluation shortcut and a parsing defect that silently disables all host features for URLs without a scheme.
2. A per-class artifact audit of six public phishing-URL corpora and three live feeds, showing that the dominant artifacts change strength and even direction between corpora.
3. A dataset-identification probe showing that the source corpus of a URL is predictable from lexical features far above chance, that dropping artifact columns hardly changes this, and that canonicalising the URL string does.
4. A controlled evaluation of random versus registered-domain-grouped splits, external-corpus and leave-one-corpus-out transfer, calibration, threshold transfer, explanations and robustness to label-preserving URL variants.
5. A tested, reproducible implementation with documented provenance for every dataset and number.

Section II reviews the literature and derives the research gap. Section III states the problem and hypotheses. Section IV describes and audits the baseline system. Section V presents the data, Section VI the methodology and Section VII the experimental setup. Section VIII reports results, Section IX discusses them, Section X lists threats to validity and Section XI concludes.

II. Literature Review

We reviewed 111 sources identified through searches of IEEE Xplore, the ACM Digital Library, SpringerLink, ScienceDirect, USENIX and NDSS proceedings, arXiv and Google Scholar, with every bibliographic record verified against an authoritative page. This section organises the most relevant work by theme; Table I compares representative studies.

A. Lexical features and classical learners

Early measurement work established that the URL string alone separates phishing from benign pages. McGrath and Gupta found phishing URLs far longer than benign directory URLs (modal lengths of 67–107 versus 22 characters) and observed brand names in 50–75 % of them [17]; Garera et al. catalogued URL obfuscation strategies such as IP hosts, redirection and brand names in the host or path [18]. Ma et al. turned these observations into large sparse bag-of-token models combined with host-based features and reported 95–99 % accuracy [3]. Their online variant, evaluated day by day over 100 days of web-mail traffic, showed that the feature distribution of malicious URLs drifts and that stale batch models degrade [19, 20]. Importantly for this study, Ma et al. also reported that training on one data source and testing on another raised error to roughly 14–21 % [3].

Later work refined the feature space: obfuscation-resistant lexical features with online AROW learning [4], confidence-weighted online learning [21], distributional character statistics [22], word-level natural-language features [6], search-engine word relatedness [23] and fast linear-time extractors [24]. Benchmarks such as ISCX-URL-2016 [5], the Hannousse–Yahiouche dataset [15] and PhiUSIIL [25]

made lexical detection easy to evaluate, and tree ensembles consistently emerged as the strongest classical learners.

B. Ensembles and hybrid detection systems

Ensembles combine heterogeneous learners or feature views. CatchPhish combined TF-IDF n-grams with handcrafted features in a random forest [26]; Chiew et al. selected compact feature subsets with an ensemble procedure [27]; PhishHaven used unbiased voting over several lexical models [28]; and Lee et al. trained noisy feature-subset ensembles to raise evasion cost [29]. The gains are consistently small: on the 651k-URL Kaggle corpus, stacking improved accuracy from 95.7 % (LightGBM) to 96.8 % [7], and on a 248k-URL feature table a Bayesian-tuned stacking ensemble matched but did not exceed Extra Trees in ROC-AUC (0.9947 versus 0.9948) [8]. These comparisons were made on random splits of a single dataset.

C. Character-level, sequence and transformer models

Learned representations avoid manual feature design. Character-level CNNs [30], character LSTMs [31] and joint character–word CNNs [32] were followed by attention-based hybrids [33, 34], Inception-style character/word networks evaluated at a 0.01 % false-positive rate on production telemetry [35], n-gram embeddings with attention [36] and multi-million-URL CNN studies [37]. Transformers fine-tuned on browser telemetry raised the true-positive rate at 0.01 % FPR from 71.2 % (URLNet) to 86.8 % [38]; URL-specific pre-training at billion-URL scale followed [39, 40, 41], as did lightweight on-device transformers [42, 43] and large-language-model prompting and fine-tuning [44, 45]. Two observations matter here. First, the studies that used temporal splits and low-FPR operating points on production data [32, 35, 38] report far more modest true-positive rates than public-benchmark papers. Second, TransURL is the only representation paper we found with a substantial cross-dataset test, in which all models degraded sharply (GramBeddings fell to 70.2 % accuracy) [41]. Aljofey et al. stripped `http://`, `https://` and `www.` before training a character CNN [46], implicitly treating these prefixes as recording artifacts, but did not measure the effect.

D. Explainability, robustness and calibration

SHAP provides additive feature attributions with consistency guarantees [47], and TreeExplainer computes them exactly for tree ensembles [48]. In phishing work SHAP is used mainly to select features [49, 50, 8]; Mia et al. are the exception, using it to show that the same features change sign across datasets [9]. Robustness studies show that 50 reproduced URL detectors fell from a median MCC of 0.92 to 0.02 on cheaply registrable adversarial URLs [16], that realistic problem-space evasions cause statistically significant drops [51], and that generated URLs evade detectors [52, 53]; practitioners report that cheap manipulations, not gradient attacks, dominate in practice [54]. Homograph and squatting domains remain effective [55, 56, 57]. On calibration, boosted trees are known to be mis-calibrated without post-hoc scaling [58], calibration assumes that deployment matches the calibration data [59], and in Android malware detection calibration degrades under dataset shift [60]. We found no comparable study for phishing URLs.

E. Evaluation realism: sampling bias, leakage, drift and base rates

Arp et al. showed that sampling bias, spurious correlations and the base-rate fallacy affect most top-venue security ML papers [12], echoing earlier warnings about closed-world evaluation [61] and base rates [62]. TESSERACT showed that temporally inconsistent splits inflate malware-detection F1 [63, 64], and drift-aware rejection and continual learning only partly recover performance [65, 66, 67]; phishing models trained on older data also degrade on newer data [68, 11]. For phishing specifically, the benign class has repeatedly been identified as the weak point: Alexa index pages "create a bias in the data" [16]; Alexa domain lists lack subdomains and paths [15]; benign sets should contain unpopular

sites and URLs with paths, at realistic prevalence [13]; balanced, popularity-derived benchmarks overstate performance [14, 69, 70]; popularity lists are unstable and manipulable [71, 72]; and public datasets leak between train and test and use unrealistic base rates [73]. Cross-dataset studies report near-chance transfer [9, 74] and propose domain adaptation [10]. Closest to our work, a single-author preprint names the `https://` and `www.` shortcut and tests URL canonicalisation against one external benign set [75], and a study of phishing *e-mail* uses a corpus-identification probe and artifact-removal ablations [76].

F. Comparative survey

TABLE I. Comparison of representative studies. Reported results are the authors' own and are not comparable across rows, because datasets, splits, class ratios and metrics differ.

Ref.	Study	Approach	Data	Evaluation	Reported result	Limitation for our question
[3]	Ma et al. (2009)	Bag-of-tokens lexical and host features; logistic regression and SVM	Yahoo/DMOZ benign vs PhishTank/Spamscatter, 20–30k URLs per set	Ten random 50/50 splits; cross-source test	95–99 % accuracy; 14–21 % error when trained and tested on different sources	Host features need live lookups; homepage-heavy benign sources
[19]	Ma et al. (2009)	Online confidence-weighted learning on lexical and host features	About 2 M web-mail URLs over 100 days	Day-ahead temporal evaluation	Up to 99 % accuracy; stale batch models degrade	Proprietary feed; not reproducible
[4]	Le et al. (2011)	Obfuscation-resistant lexical features with AROW online learning	About 4k PhishTank/MalwarePatrol vs Yahoo/DMOZ URLs per set	Online cumulative error	Lexical-only 96–98 % accuracy, within 1 % of full features	Small sets; directory homepages as benign
[5]	Mamun et al. (2016)	Lexical features with tree and kNN classifiers	ISCX-URL-2016, about 110k URLs in five classes	Per-class and multi-class evaluation	About 99 % per class; 93–99 % multi-class (random forest)	Each class comes from a different source
[23]	Marchal et al. (2014)	Word relatedness from search-engine query data	96,018 URLs: PhishTank vs DMOZ	10-fold cross-validation	94.91 % accuracy at 1.44 % FPR	Depends on search-engine APIs; homepage-heavy benign class
[6]	Sahingoz et al. (2019)	Word-level NLP features; seven classifiers	Ebbu2017: 73,575 URLs	Hold-out split	97.98 % accuracy (random forest)	2017 data; no cross-dataset test
[15]	Hannousse et al. (2021)	Benchmark construction guidelines and classifier comparison	11,430 URLs with 87 features	Random stratified split	About 96.6 % accuracy (random forest, all features)	Small and balanced; authors find results dataset-dependent
[7]	Sankaranarayanan et al. (2024)	Stacking of boosted and bagged trees on lexical features	Kaggle corpus, 651,191 URLs in four classes	Stratified 80/20 split	Stacking 96.8 % vs LightGBM 95.7 % accuracy	Mixed-source corpus; single random split
[8]	Khan et al. (2026)	Bayesian-optimised stacking with SHAP feature selection	247,950 Kaggle URLs (pre-extracted features)	Stratified 80/20 with internal 5-fold CV	ROC-AUC 0.9947 (stacking) vs 0.9948 (Extra Trees)	No raw URLs; single dataset
[30]	Saxe et al. (2017)	Character-level CNN	Proprietary VirusTotal-derived URLs	Held-out split, low-FPR ROC	5–10 % higher detection than manual features at 0.1 % FPR	Not peer reviewed; proprietary data
[31]	Bahnsen et al. (2017)	Character LSTM vs random forest on engineered features	2 M URLs: PhishTank vs Common Crawl	3-fold cross-validation	98.7 % (LSTM) vs 93.5 % (random forest) accuracy	Random folds; benign and phishing from different sources
[32]	Le et al. (2018)	Joint character- and word-level CNN	About 15 M VirusTotal URLs	Time-ordered 60/40 split; TPR at fixed FPR	Best AUC; higher TPR than bag-of-words at FPR 10^{-3}–10^{-1}	Labels from blacklist counts
[35]	Tajaddodianfar et al. (2020)	Parallel character/word CNN	Microsoft telemetry; 20 M test URLs	Temporal split; TPR at 0.01 % FPR	126.7 % relative TPR gain over a text classifier	Proprietary data
[46]	Aljofey et al. (2020)	Character-level CNN with scheme and www. removed	318,642 URLs plus three benchmarks	Random splits	95.0 % (main set); 95.2–98.6 % on benchmarks	Effect of prefix removal not measured
[38]	Maneriker et al. (2021)	Fine-tuned BERT/RoBERTa (URLTran)	Six weeks of browser telemetry, 1:20 class ratio	Temporal split; TPR at 0.01 % FPR	86.8 % TPR vs 71.2 % for URLNet at 0.01 % FPR	Proprietary data
[41]	Liu et al. (2024)	Character-aware transformer with pyramid features	GramBeddings, Mendeley and two Kaggle corpora	Random, cross-dataset and adversarial tests	Cross-dataset accuracy 90.2 % vs 70.2 % for GramBeddings	Cross-dataset results far below in-distribution
[40]	Li et al. (2025)	URL-pretrained BERT (3.28 B URLs)	364,622 phishing and benign URLs	60/40 split; TPR at 0.1 % FPR	ROC-AUC above 0.99; classical models still above 96 % accuracy	URLs longer than 150 characters removed
[16]	Sabir et al. (2022)	Reproduction of 50 detectors plus an adversarial URL generator	96,693 phishing URLs and crawled Alexa benign URLs	Baseline vs adversarial URLs (MCC)	Median MCC falls from 0.92 to 0.02	Balanced, random-split baseline
[51]	Apruzzese et al. (2022)	Problem-space vs feature-space evasion of 18 detectors	Two public phishing-website datasets	Repeated trials with significance tests	3–10 % significant detection drop from cheap evasions	Website-level detectors
[9]	Mia et al. (2024)	Within- and cross-dataset training with SHAP	Feature tables of 88,647 and 19,431 URLs	Random splits; cross-dataset transfer	Cross-dataset accuracy of 51 % and 59 %	Pre-computed features, so raw-URL artifacts cannot be ablated
[10]	Rashid et al. (2024)	Unsupervised domain adaptation	Three phishing-URL datasets	Pairwise cross-dataset F1	F1 gains of 0.06 on average (up to 0.2)	Mechanism of the shift not isolated

Ref.	Study	Approach	Data	Evaluation	Reported result	Limitation for our question
[11]	Sanchez-Paniagua et al. (2022)	Benign login URLs as a harder benign class	PILU-90K: 30k homepages, 30k login URLs, 30k phishing	Homepage-trained models tested on login URLs; 2016→2020	27–39 % accuracy loss on login URLs	Random splits within each period
[75]	Giri (2026)	URL canonicalisation and rule fusion	Public sets; OpenPhish and Umbrella snapshots externally	Frozen model, domain-disjoint external test	86.84 % FPR on an external benign set	Single-author preprint; one model family
[12]	Arp et al. (2022)	Systematic study of ten pitfalls of ML in security	30 top-venue papers and four case studies	Prevalence analysis and controlled experiments	Sampling bias and spurious correlations are common	No phishing-URL case study
[63]	Pendlebury et al. (2019)	Time-aware evaluation framework (TESSERACT)	About 129k Android apps, 2014–2016	Temporal splits; area under time	AUT(F1, 24 months) 0.32–0.64, below 10-fold scores	Malware rather than URLs
[60]	Li et al. (2021)	Calibration and uncertainty under dataset shift	Android malware detectors	Temporal shift and adversarial examples	Calibration degrades under shift and fails under attack	Malware rather than URLs
[76]	Bhuiyan et al. (2026)	Corpus-identification probe and artifact-removal ablation	Six phishing and spam e-mail corpora	Leave-one-corpus-out; prevalence shift	Source corpus identified with 97–98 % accuracy	E-mail text; linear models only
[73]	Dalton et al. (2025)	Leakage-pruned, temporally split phishing benchmark	Large crawled phishing-website dataset	Temporal split; base rates 0.05–5 %	Shows public datasets leak and use unrealistic base rates	Preprint; HTML-focused

G. Research gap

The literature establishes that lexical detectors score highly in-distribution, that they transfer poorly between datasets, and that popularity-list benign data are biased. It does not establish *which* recording properties of public corpora carry the in-distribution scores, whether those properties point the same way across corpora, how much registered-domain memorisation adds, or whether removing the properties — by dropping features or by canonicalising the URL — restores transfer. Prior cross-dataset studies worked with pre-computed feature tables [9, 74], which makes it impossible to ablate raw-string artifacts, or did not isolate the mechanism [10]; the one direct treatment is an unreviewed single-model preprint [75]. Calibration and threshold transfer have not been studied for phishing URLs, and robustness work has focused on adversarial manipulation rather than on label-irrelevant recording variants such as a missing scheme or a `www.` prefix. These gaps define the study below.

III. Research Problem and Hypotheses

A detector maps a URL string u to a score $p(u)$ that estimates P(malicious | u) and flags u when $p(u) \geq \tau$. We call a property $a(u)$ a **collection artifact** if it is determined by how a source recorded or sampled URLs rather than by the URL's intent, and if its distribution differs between the classes of a corpus. The scheme prefix (absent, `http` or `https`), a leading `www.` label, and the restriction of a benign source to bare homepages satisfy both conditions in the corpora studied (Section V).

We test three hypotheses:

- **H1** (*artifact reliance*). Removing artifact information changes in-distribution ROC-AUC only marginally but changes behaviour on external corpora substantially; the source corpus of a URL is identifiable from lexical features well above chance; and random splits over-estimate grouped-split performance when corpora contain many URLs per registered domain.
- **H2** (*operating points*). Thresholds selected for a 1 % false-positive budget on validation data do not hold on external corpora, and post-hoc calibration does not repair this.
- **H3** (*temporal recall*). Recall on timestamped phishing feeds changes with submission year as the share of https phishing rises.

IV. The Baseline System and Its Audit

A. System description

The baseline system extracts 28 lexical features from each URL: lengths and counts of dots, slashes, hyphens, underscores, digits and special characters; the digit-to-letter ratio; path, query and domain lengths; the number of query parameters, `@` symbols and `//` occurrences; Shannon entropy; a subdomain count; the last hostname label (`tld_type`); flags for suspicious TLDs, punycode, IP hosts, `https`, URL shorteners, suspicious keywords, brand names, excessive hyphens and "random-looking" domains. `tld_type` is one-hot encoded (521 columns on the original data). Three classifiers are trained — logistic regression (standardised inputs, balanced class weights), a random forest of 200 trees and XGBoost with 200 rounds of depth 6 — and combined by soft voting with weights 1:2:2. The data are split 60/20/20 at random with stratification and seed 42; predictions use a fixed threshold of 0.5. A Flask application serves single-URL predictions, and SHAP KernelExplainer explains the ensemble.

B. Audit findings

We read every file of the repository and extracted the stored outputs of its nine notebooks. Four findings shape this study. (i) The dataset (`raw_data.csv`) is not part of the repository and its origin is undocumented; our provenance search traced four of its five visible rows to a Kaggle/HuggingFace URL lineage built from ISCX-URL-2016, PhishTank and PhishStorm, stored there *without* a scheme, whereas the project file prefixes `https://www.` — evidence of a normalisation step applied to the benign class. (ii) The stored SHAP importances are dominated by `https_usage` (0.156) and `num_subdomains` (0.091), and in the stored class means 93 % of legitimate but only 50 % of phishing URLs use https. (iii) The weighted ensemble does not outperform its own random forest (test F1 0.9665 versus 0.9668). (iv) The feature extractor parses no host when a URL has no scheme: `visitcanada.com/` receives a domain length of 0 and the TLD "ip", so all host features collapse to constants for such input. The deployed application also passed a 28-column frame to a model trained on 548 one-hot columns and could not run as committed.

C. Faithful re-execution

Because the original data are unavailable, we re-executed the unmodified pipeline code on two substitute corpora (Section V). Table II compares the result with the stored numbers.

TABLE II. Re-execution of the original pipeline code (random 60/20/20 split, seed 42, threshold 0.5) on two substitute corpora, compared with the stored results on the unavailable original dataset.

Data	Model	Accuracy	Precision	Recall	F1	ROC-AUC
Original data (stored)	LR	0.8864	0.9030	0.8598	0.8809	not stored
Original data (stored)	RF	0.9679	0.9771	0.9568	0.9668	not stored
Original data (stored)	XGBoost	0.9671	0.9828	0.9492	0.9657	not stored
Original data (stored)	Soft vote 1:2:2	0.9678	0.9824	0.9512	0.9665	not stored
PhiUSIIL (re-run)	LR	0.9952	0.9985	0.9903	0.9944	0.9983
PhiUSIIL (re-run)	RF	0.9968	0.9984	0.9940	0.9962	0.9984
PhiUSIIL (re-run)	XGBoost	0.9969	0.9992	0.9936	0.9964	0.9986
PhiUSIIL (re-run)	Soft vote 1:2:2	0.9970	0.9994	0.9935	0.9965	0.9988
Corpus B (re-run)	LR	0.7025	0.7403	0.6009	0.6633	0.805
Corpus B (re-run)	RF	0.8455	0.8465	0.8346	0.8405	0.9239
Corpus B (re-run)	XGBoost	0.8406	0.8542	0.8117	0.8324	0.9265
Corpus B (re-run)	Soft vote 1:2:2	0.8484	0.8529	0.8328	0.8427	0.9317

On PhiUSIIL the code scores higher than the project reported on its own data (soft-vote accuracy 0.9970, ROC-AUC 0.9988), and the four models lie within 0.0018 accuracy of one another. On corpus B, 99.8 % of whose URLs lack a scheme, accuracy falls to 0.8484: the parsing defect, not the corpus, limits the baseline there.

V. Data

A. Corpora

We assembled six labelled corpora with raw URL strings and three timestamped or live feeds (Table III). Corpus A is PhiUSIIL [25]. Corpus B is a 253,098-URL stratified sample of the Kaggle/HuggingFace lineage, sized and balanced exactly like the unavailable original dataset (129,420 benign, 123,678 phishing). The external labelled corpora are Hannousse and Yahiouche 2021 [15], Ebbu2017 [6], PhishStorm [23] and the benign and phishing lists of ISCX-URL-2016 [5]; the 651k-URL Kaggle corpus [77] was scored in the first two runs only because it shares lineage with corpus B. The positive-only feeds are the PhishTank verified-online dump with submission times [78], URLhaus malware URLs [79] and an OpenPhish snapshot [80].

Every source is mapped to a URL and a label with malicious = 1. Duplicates are detected on a normalised key (lower-case scheme and host, default port and fragment removed, empty path collapsed); URLs carrying both labels are removed; features are always computed on the original string. Each URL is assigned a group equal to its registered domain, computed with an offline Public Suffix List snapshot, or its IP address.

TABLE III. Corpora after cleaning (malicious = 1). Shares are over all URLs of a corpus; PhishTank, URLhaus and OpenPhish contain only malicious URLs.

Corpus	Role	Raw	Kept	Malicious	Benign	Reg. domains	https	www.	Timestamps
PhiUSIIL	training (A)	235,795	234,687	99,838	134,849	175,508	0.78	0.75	no
Corpus B	training (B)	253,098	252,394	123,095	129,299	130,453	0.00	0.16	no
Hannousse 2021	external	11,430	11,416	5,701	5,715	6,634	0.39	0.44	no
Ebbu2017	external	73,575	54,721	24,811	29,910	21,606	0.21	0.44	no
PhishStorm	external	96,018	95,962	47,972	47,990	30,894	0.02	0.52	no
ISCX-2016	external	45,343	45,324	9,946	35,378	4,304	0.05	0.10	no
Kaggle 651k	external*	522,214	514,031	90,015	424,016	145,482	0.02	0.10	no
PhishTank	feed	67,621	67,326	67,326	0	12,971	0.92	0.04	2011-02 to 2026-10
URLhaus	feed (malware)	72,849	72,847	72,847	0	17,585	0.38	0.01	2018-06 to 2026-10
OpenPhish	feed	300	300	300	0	144	0.71	0.19	no

B. Collection artifacts

Fig. 1 reports, per corpus and class, the share of URLs with an https scheme, a leading `www.` label and an empty path. In PhiUSIIL every benign URL is an `https://www.` homepage with no path, whereas 48 % of phishing URLs use https, 42 % start with `www.` and 72 % are homepages. The same properties behave very differently elsewhere. Corpus B and the Kaggle corpus store no scheme at all; in ISCX-2016 and the Kaggle corpus `www.` is *more* common among phishing URLs than among benign ones; and ISCX-2016's 35,378 benign URLs are deep pages from only about 300 sites. The present-day phishing population differs again: 92 % of PhishTank URLs use https and only 4 % start with `www.`. Measured as single-feature ROC-AUC (values below 0.5 indicate association with the benign class), the `www.` prefix scores 0.21 in PhiUSIIL, 0.27–0.34 in Hannousse, Ebbu2017 and PhishStorm, but 0.71 in ISCX-2016 and 0.69 in the Kaggle corpus; URL length scores 0.72–0.73 in PhiUSIIL and PhishStorm and 0.24 in ISCX-2016.

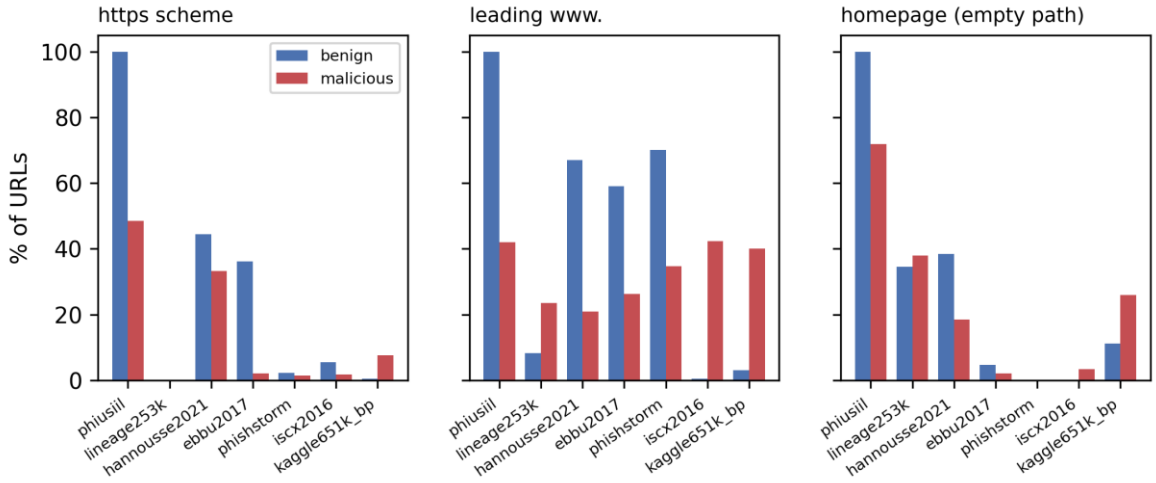


Fig. 1. Share of URLs with an https scheme, a leading www. label and an empty path, by class and corpus. Corpus B and the Kaggle corpus store URLs without a scheme.

VI. Methodology

Fig. 2 summarises the pipeline. All fitting uses training rows only; every model, threshold and calibration choice uses validation rows only; test partitions, external corpora and feeds are scored once.

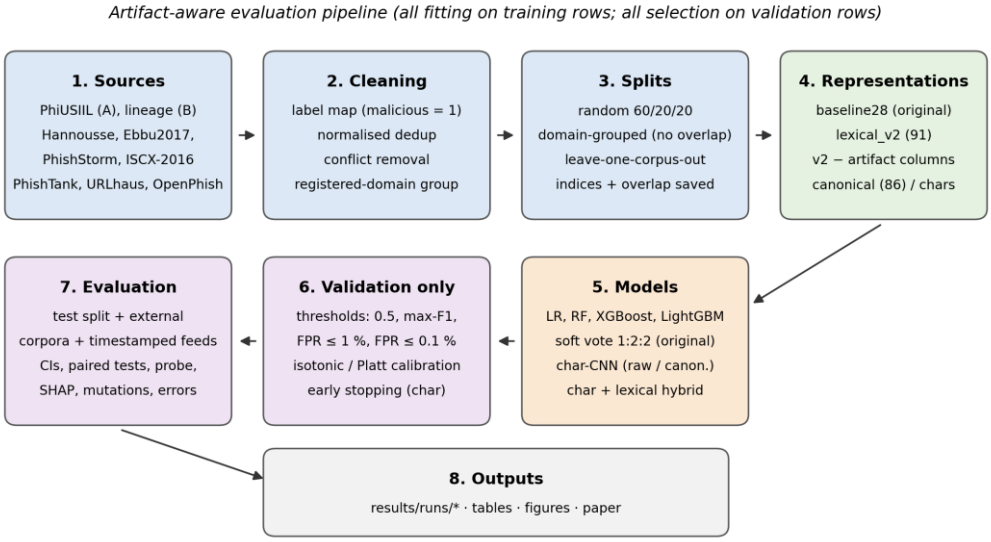


Fig. 2. Artifact-aware evaluation pipeline.

A. URL parsing

URLs are untrusted input, so the parser accepts any string and never raises. When no scheme is present it parses the string as `http://` + URL and records that the scheme was absent; malformed authorities and invalid ports set a parse-error flag. It extracts the scheme, user information, host, port, path, query, fragment, subdomain, registered domain, public suffix, IP and IPv6 status and the IDNA-decoded host. Unit tests cover missing schemes, IPv4 and IPv6 literals, ports, user information, punycode, Unicode hosts, single and double percent-encoding, very long strings and non-URL input.

B. Representations

We compare four handcrafted representations:

- **baseline28** — the original 28 features computed by the original, unmodified code; `tld_type` is one-hot encoded with the encoder fitted on training rows only and TLDs with fewer than 20 occurrences pooled.
- **lexical_v2** — 91 offline features computed from the parsed components: whole-URL shape (length, entropy, character-class counts and ratios, longest digit run, token statistics, hexadecimal tokens, double encoding); scheme and authority (https, explicit scheme, `www.` prefix, user information, ports); host structure (lengths and entropy, label counts with and without the public suffix, registered-domain length, digits and hyphens, consonant runs, IP, punycode and non-ASCII flags, abused suffixes, free-hosting suffixes, shorteners); brand tokens *placed* in subdomain, registered domain or path rather than matched as raw substrings; and path, query and fragment structure.
- **lexical_v2 − artifacts** — lexical_v2 without the five columns that encode recording format (https, explicit scheme, `www.` prefix and two label counts); the analogous ablation of baseline28 removes `https_usage` and `num_subdomains`.
- **lexical_v2_canon** — the URL string is first *canonicalised* by removing the scheme and one leading `www.`/`wwwN.` label; lexical_v2 is computed on the result and the five artifact columns are dropped, leaving 86 features. Every feature is therefore invariant to the presence or form of the scheme and the `www.` prefix, a property verified by unit tests.

C. Models

We evaluate the baseline's four models with its own hyper-parameters, so that every comparison is faithful to the original system: logistic regression (standardised inputs, C = 1, balanced class weights), a random forest of 200 trees with balanced class weights, XGBoost with 200 rounds of depth 6, learning rate 0.1 and row and column subsampling of 0.8, and the soft vote that averages their probabilities with weights 1:2:2. No hyper-parameter is tuned on test or external data. Linear and tree models were chosen because they dominate published lexical detectors (Section II-A, II-B) and expose the decision mechanism through coefficients and exact tree SHAP values.

D. Evaluation protocols

We use three split protocols. **Random**: stratified 60/20/20. **Grouped**: ten stratified folds grouped by registered domain, of which two form the test set, two the validation set and six the training set, so that no registered domain appears in two partitions. **Leave-one-corpus-out (LOCO)**: for each of the six labelled corpora, the other five are pooled (each capped at 50,000 URLs, stratified), split by registered domain into training, validation and test partitions, and the entire held-out corpus becomes the evaluation set. In addition, models trained on corpus A or B are applied unchanged to every other corpus and feed (**external** evaluation).

E. Operating points and calibration

For every model and seed, four thresholds are fixed on validation data: 0.5; the threshold maximising F1; and the most sensitive thresholds whose validation false-positive rate is at most 1 % and 0.1 %. The same thresholds are applied unchanged to test partitions and external corpora. Isotonic regression and Platt scaling are fitted on validation scores to test post-hoc calibration.

F. Analyses

Probe. For each class separately, up to 8,000 URLs per corpus are pooled from the six labelled corpora and a multinomial logistic regression predicts the source corpus, under five-fold cross-validation grouped by registered domain (chance 1/6). **Explanations.** TreeExplainer SHAP values are computed for XGBoost on 3,000 test URLs. **Label-preserving variants.** *Tier A* transformations are equivalent under RFC 3986 and preserve the label by construction: upper-casing scheme and host, adding an empty-path slash, adding the default port, percent-encoding one unreserved path character and adding a fragment.

Tier B transformations change only presentation and are *assumed* to preserve the label: toggling a leading `www.`, toggling `http`/`https` and appending a tracking parameter. We report how often the decision at 0.5 flips. **Errors.** False positives and false negatives are stratified by domain novelty, scheme, `www.`, length, path depth, encoding and hosting type.

G. Metrics and statistics

Malicious URLs are the positive class. We report accuracy, precision, recall, specificity, F1, false-positive and false-negative rates, ROC-AUC, PR-AUC, the true-positive rate at 1 % and 0.1 % FPR, log-loss, Brier score and expected calibration error (ECE, 15 bins). On positive-only feeds only recall is defined. Results are averaged over seeds and reported as mean ± standard deviation; single-seed estimates carry 95 % percentile-bootstrap intervals; comparisons on identical test URLs use a paired bootstrap of metric differences and McNemar's exact test. A difference is called significant only if its paired 95 % interval excludes zero and its sign agrees across seeds.

VII. Experimental Setup

All experiments ran on an Intel Core i7-1355U laptop processor (10 cores) with 16 GB of RAM and no GPU, under Windows 11 with Python 3.10.11, scikit-learn 1.7.2, XGBoost 3.2.0, SHAP 0.49.1 and tldextract 5.3.1. SHAP values for XGBoost are computed with XGBoost's built-in exact TreeSHAP. Runs on corpus A use seeds 42–46 and LOCO runs seeds 42–44. Each run stores its configuration, software environment, split indices with a domain-overlap report, per-URL predictions, metrics and resource measurements, so that every number in this paper can be regenerated. Table IV lists the runs reported and the number of seeds behind each.

TABLE IV. Experiment runs reported in this paper. Seeds = completed random seeds per model; every number reported is a mean over these seeds.

Run	Description	Models	Seeds	Evaluated on (besides own test split)
A_b28_random	Corpus A, original features, random split	LR, RF, Soft vote 1:2:2, XGBoost	5	Ebbu2017, Hannousse 2021, ISCX-2016, Kaggle 651k, Corpus B, OpenPhish, PhishStorm, PhishTank, URLhaus
A_b28_grouped	Corpus A, original features, grouped split	LR, RF, Soft vote 1:2:2, XGBoost	5	Ebbu2017, Hannousse 2021, ISCX-2016, Kaggle 651k, Corpus B, OpenPhish, PhishStorm, PhishTank, URLhaus
A_b28noart_grouped	Corpus A, original features minus artifact columns, grouped	LR, RF, Soft vote 1:2:2, XGBoost	4–5	Ebbu2017, Hannousse 2021, ISCX-2016, Corpus B, OpenPhish, PhishStorm, PhishTank, URLhaus
LOCO_phiusiil_v2	LOCO, PhiUSIIL held out, lexical_v2	LR, RF, XGBoost	3	PhishTank, PhiUSIIL
LOCO_phiusiil_v2noart	LOCO, PhiUSIIL held out, lexical_v2 minus artifact columns	LR, RF, XGBoost	3	PhishTank, PhiUSIIL
LOCO_lineage253k_v2	LOCO, corpus B held out, lexical_v2	LR, RF, XGBoost	3	Corpus B, PhishTank
LOCO_lineage253k_v2noart	LOCO, corpus B held out, lexical_v2 minus artifact columns	LR, RF, XGBoost	3	Corpus B, PhishTank
LOCO_hannousse2021_v2	LOCO, Hannousse 2021 held out, lexical_v2	LR, RF, XGBoost	3	Hannousse 2021, PhishTank
LOCO_hannousse2021_v2noart	LOCO, Hannousse 2021 held out, lexical_v2 minus artifact columns	LR, RF, XGBoost	3	Hannousse 2021, PhishTank

VIII. Results

A. The source corpus of a URL is learnable (RQ1, RQ2)

If collection artifacts differ between corpora, a classifier should be able to tell corpora apart from lexical features alone. Table V confirms this. With lexical_v2, a linear probe identifies which of six corpora a **benign** URL comes from with accuracy 0.830, against a chance level of 0.167, and a malicious URL with accuracy 0.606. Dropping the five artifact columns barely changes this (0.814 and 0.578), because the scheme still leaks through slash, colon and length counts: the two scheme-less corpora remain identifiable with accuracy of 0.97 or more. Canonicalising the URL string before computing features

lowers the accuracies to 0.617 and 0.428 and makes the scheme-less corpus B almost indistinguishable among benign URLs (0.184). PhiUSIIL's benign URLs remain identifiable at 0.981 even after canonicalisation, because being a bare homepage is a sampling property that no string transformation can remove. The baseline's own 28 features identify the source corpus with accuracy 0.803 (benign) and 0.577 (malicious).

TABLE V. Dataset-identification probe: accuracy of a six-way logistic regression predicting the source corpus of a URL (grouped five-fold cross-validation; chance = 0.167), overall and recall per corpus.

Class	Representation	Accuracy	PhiUSIIL	Corpus B	Hannousse	Ebbu2017	PhishStorm	ISCX-2016
benign	original 28 features	0.803	1.000	0.999	0.367	0.544	0.841	0.945
benign	lexical_v2	0.830	1.000	1.000	0.402	0.670	0.820	0.967
benign	lexical_v2 minus artifact columns	0.814	1.000	0.990	0.383	0.661	0.766	0.961
benign	lexical_v2_canon (canonicalised)	0.617	0.981	0.184	0.319	0.553	0.703	0.874
malicious	original 28 features	0.577	0.872	0.999	0.180	0.506	0.300	0.489
malicious	lexical_v2	0.606	0.865	0.999	0.277	0.535	0.385	0.481
malicious	lexical_v2 minus artifact columns	0.578	0.839	0.972	0.179	0.544	0.369	0.450
malicious	lexical_v2_canon (canonicalised)	0.428	0.748	0.344	0.107	0.505	0.370	0.402

B. In-distribution performance and the role of domain leakage (RQ1)

Random splitting leaks registered domains asymmetrically. With seed 42, 59.4 % of the phishing test URLs of corpus A share a registered domain with some training URL, against only 2.4 % of the benign test URLs (26.7 % overall); grouped splitting removes this overlap by construction. Despite the leakage, grouping hardly changes in-distribution performance (Table VI): every model reaches a ROC-AUC of 0.998–0.999 under both protocols, and F1 drops by at most 0.002. The models do not need memorised domains, because the collection artifacts already separate the classes.

TABLE VI. In-distribution test performance on corpus A with the original 28 features: random vs registered-domain-grouped split (mean ± sd over 5 seeds).

Model	ROC-AUC random	ROC-AUC grouped	F1@0.5 random	F1@0.5 grouped
Logistic regression	0.998 ± 0.000	0.998 ± 0.000	0.995 ± 0.001	0.993 ± 0.003
Random forest	0.998 ± 0.000	0.998 ± 0.000	0.996 ± 0.001	0.994 ± 0.003
XGBoost	0.999 ± 0.000	0.999 ± 0.000	0.996 ± 0.001	0.995 ± 0.003
Soft vote 1:2:2	0.999 ± 0.000	0.999 ± 0.000	0.996 ± 0.001	0.995 ± 0.003

The differences between models are tiny. Paired on identical test URLs (Table VII), XGBoost is significantly better than logistic regression and random forest, but only by 0.0003–0.0004 ROC-AUC. The original 1:2:2 soft-voting ensemble is *not* significantly better than XGBoost alone (ΔROC-AUC = +0.0001, 95 % CI [−0.0000, 0.0002]; McNemar p = 0.21). The ensemble, the original system's main design choice, therefore adds cost but no measurable accuracy.

TABLE VII. Paired comparisons against XGBoost on the 48,181 grouped test URLs of corpus A (seed 42): bootstrap difference in ROC-AUC and F1 with 95 % confidence interval, and McNemar's exact test on paired decisions.

Model vs XGBoost	Δ ROC-AUC [95 % CI]	Δ F1 [95 % CI]	Only model / only XGBoost correct	McNemar p
LR	-0.00034 [-0.00061, -0.00008]	-0.0017 [-0.0021, -0.0012]	20 / 90	< 0.001
RF	-0.00040 [-0.00065, -0.00018]	-0.0004 [-0.0007, -0.0001]	18 / 35	0.027
Soft vote 1:2:2	+0.00009 [-0.00001, +0.00020]	+0.0001 [-0.0000, +0.0003]	11 / 5	0.210

The explanations show what the models rely on. For XGBoost trained on corpus A, the mean absolute SHAP value of `https_usage` is 3.65 log-odds, followed by `num_slashes` (2.69), which in this corpus mostly records whether a URL has a path; every other feature contributes less than 0.45 (Fig. 3). In other words, the model has learnt that a URL with an https scheme and no path is benign. A model trained on the five pooled corpora still ranks the scheme first, but with a far smaller weight (0.74) and a flatter profile (`num_slashes` 0.65, `num_hyphens` 0.61, longest token 0.47).

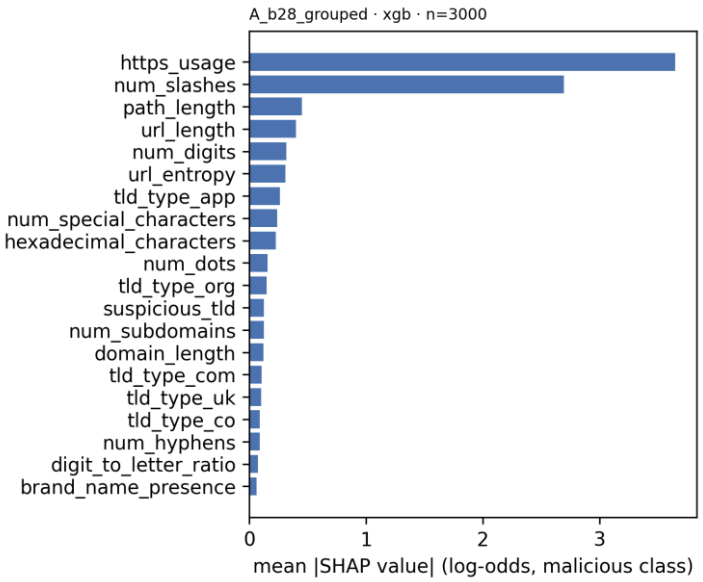


Fig. 3. Mean absolute SHAP value (log-odds of the malicious class) of the 20 most important features for XGBoost trained on corpus A with the original features (3,000 grouped test URLs).

The few in-distribution errors follow the same pattern. At the 1 % validation-FPR threshold, 70 % of the missed phishing URLs start with `www.`, although such URLs are only 37 % of the phishing test set: phishing URLs that look like the benign class are the ones the model misses. Conversely, every benign URL under a commonly abused suffix is flagged (11 of 11), and benign URLs of 31–50 characters are flagged 3.6 times as often as URLs of at most 30 characters (2.3 % vs 0.6 %).

C. External corpora (RQ2, H1, H2)

Models trained on corpus A were applied unchanged to every other corpus, using the threshold selected on corpus-A validation data for a 1 % false-positive budget (Fig. 4, Table VIII). In-distribution the threshold holds: the realised test FPR is 0.9–1.0 % at a recall of 0.992–0.995. On every external labelled corpus it fails. **99.4–100 % of the benign URLs of Hannousse 2021, Ebbu2017, PhishStorm and ISCX-2016 are flagged**, as are 74 % (logistic regression) to 100 % (tree models and soft vote) of corpus B's benign URLs. Ranking quality collapses as well: external ROC-AUC lies between 0.56 and 0.76 on Hannousse, Ebbu2017 and PhishStorm and falls *below chance* on ISCX-2016 (0.42–0.47) and the Kaggle corpus (0.35–0.48), the two corpora in which `www.` marks phishing rather than benign URLs (Section V-B). The in-distribution ranking of models says nothing about their external ranking: random forest, tied for best in-distribution, has the lowest external ROC-AUC on Hannousse, Ebbu2017 and PhishStorm (0.56–0.62).

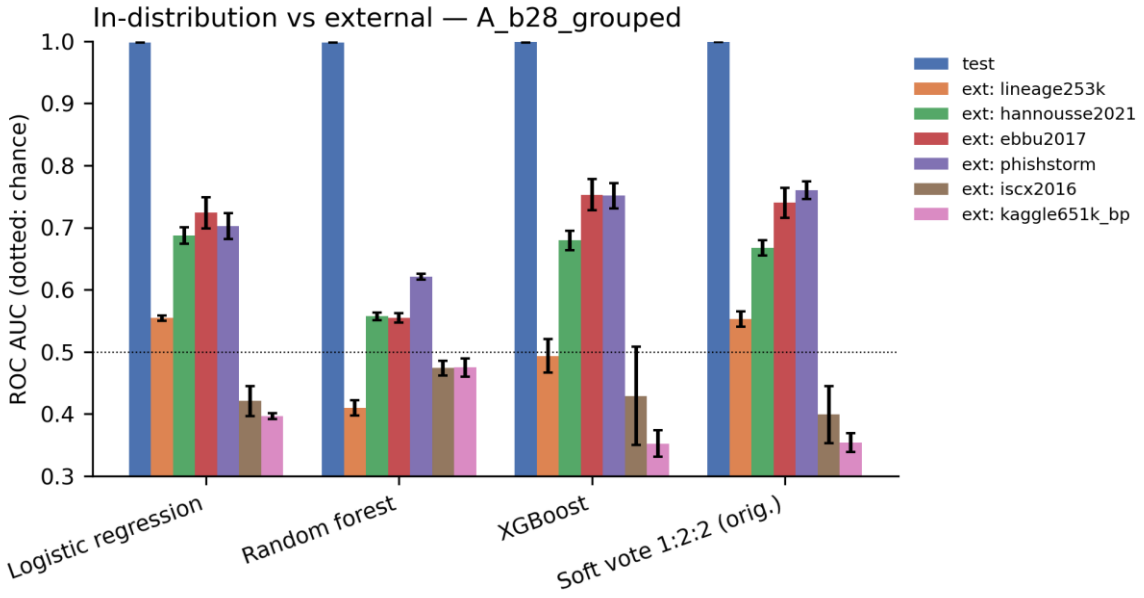


Fig. 4. ROC-AUC on the corpus-A grouped test split and on external labelled corpora for models trained on corpus A with the original features (mean ± sd over 5 seeds). Dotted line: chance.

TABLE VIII. External evaluation of corpus-A models (original features, grouped split): ROC-AUC and the share of benign URLs flagged (FPR) at the threshold chosen on corpus-A validation data for FPR ≤ 1 %; PhishTank contains only phishing URLs (recall). Cells: ROC-AUC / FPR.

Model	Test ROC-AUC	Hannousse 2021	Ebbu2017	PhishStorm	ISCX-2016	Corpus B	PhishTank recall
LR	0.998	0.688 / 0.994	0.724 / 1.000	0.703 / 1.000	0.421 / 1.000	0.554 / 0.743	0.992
RF	0.998	0.557 / 0.994	0.555 / 1.000	0.621 / 1.000	0.474 / 1.000	0.410 / 1.000	0.995
XGBoost	0.999	0.679 / 0.994	0.753 / 1.000	0.752 / 1.000	0.429 / 1.000	0.493 / 1.000	0.995
Soft vote 1:2:2	0.999	0.667 / 0.994	0.738 / 1.000	0.758 / 1.000	0.425 / 1.000	0.553 / 1.000	0.995

Recall on the positive-only feeds is 0.99–1.00 (PhishTank 0.992–0.995, OpenPhish 1.000, URLhaus 1.000). Together with the external false-positive rates, this describes a detector that flags essentially every URL not shaped like an `https://www.` homepage. At an operational prevalence of 1 % malicious URLs, a recall of 0.995 combined with an FPR of 0.994 gives a precision of about 1 %, no better than flagging at random. The models trained with a random split behave identically (external FPR 0.994–1.000 on the four independent corpora; external ROC-AUC 0.34–0.78).

D. Removing artifact features (H1)

Removing the two artifact columns of the original features (`https_usage` and `num_subdomains`) leaves in-distribution ROC-AUC almost unchanged (0.998 for XGBoost and the soft vote, 0.997 for random forest, 0.981 for logistic regression) and does not repair external behaviour (Table IX). 95–100 % of the benign URLs of the four independent corpora are still flagged, and the ROC-AUC of XGBoost on ISCX-2016 falls further, to 0.11. The shortcut survives because the remaining length and slash features still encode the homepage-only benign class, as the probe predicted. The only notable improvement is for logistic regression on the scheme-less corpus B, whose FPR falls from 0.74 to 0.24, because the linear model can no longer shift all scheme-less URLs at once.

TABLE IX. Effect of removing the artifact columns (https_usage, num_subdomains) from the original features: external ROC-AUC and FPR at the validation-chosen 1 % FPR threshold (corpus A, grouped split; mean over completed seeds). Each cell: value with all 28 features → value without the two artifact columns; first line ROC-AUC, second line FPR.

Model	Test ROC-AUC	Hannousse 2021	Ebbu2017	PhishStorm	ISCX-2016	Corpus B
LR	0.998 → 0.981	0.688 → 0.651 0.994 → 0.947	0.724 → 0.634 1.000 → 0.996	0.703 → 0.563 1.000 → 1.000	0.421 → 0.518 1.000 → 0.989	0.554 → 0.530 0.743 → 0.265
RF	0.998 → 0.997	0.557 → 0.552 0.994 → 0.982	0.555 → 0.533 1.000 → 1.000	0.621 → 0.639 1.000 → 1.000	0.474 → 0.473 1.000 → 1.000	0.410 → 0.409 1.000 → 0.996
XGBoost	0.999 → 0.999	0.679 → 0.654	0.753 → 0.527	0.752 → 0.689	0.429 → 0.115	0.493 → 0.416

Model	Test ROC-AUC	Hannousse 2021	Ebbu2017	PhishStorm	ISCX-2016	Corpus B
		0.994 → 0.994	1.000 → 1.000	1.000 → 1.000	1.000 → 1.000	1.000 → 0.759
Soft vote 1:2:2	0.999 → 0.998	0.667 → 0.649 0.994 → 0.993	0.738 → 0.554 1.000 → 1.000	0.758 → 0.722 1.000 → 1.000	0.425 → 0.161 1.000 → 1.000	0.553 → 0.425 1.000 → 0.815

E. Training on diverse corpora: leave-one-corpus-out (RQ3)

Training on five pooled corpora changes the failure mode (Fig. 5, Table X). Ranking transfers far better: with lexical_v2, the held-out ROC-AUC of XGBoost reaches 0.977 with PhiUSIIL held out, 0.927 with Hannousse 2021 held out and 0.874 with corpus B held out. Thresholds, however, do not transfer reliably, and they err in *different directions* for different unseen corpora. With PhiUSIIL held out, the threshold chosen for a 1 % false-positive budget flags no benign URL but catches only 15–42 % of phishing. With corpus B held out it flags 16 % (XGBoost), 18 % (random forest) and 95 % (logistic regression) of benign URLs. With Hannousse held out it lands close to the budget for logistic regression (1.2 %) and 6–8 times above it for the tree models.

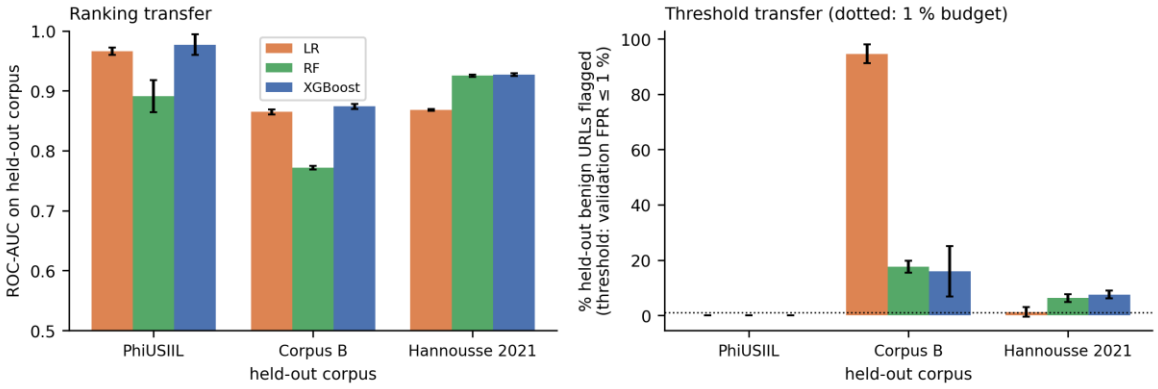


Fig. 5. Leave-one-corpus-out transfer (lexical_v2, mean ± sd over seeds): ROC-AUC on the held-out corpus (left) and the share of held-out benign URLs flagged at the threshold chosen on pooled validation data for FPR ≤ 1 % (right).

TABLE X. Leave-one-corpus-out results: models trained on the other five corpora and evaluated on the held-out corpus. Recall and FPR use the threshold chosen on pooled validation data for FPR ≤ 1 %.

Held-out corpus	Representation	Model	Seeds	Pool test AUC	Held-out AUC	Held-out recall	Held-out FPR	PhishTank recall
PhiUSIIL	lexical_v2	LR	3	0.921	0.966	0.147	0.000	0.159
PhiUSIIL	lexical_v2	RF	3	0.960	0.891	0.251	0.000	0.293
PhiUSIIL	lexical_v2	XGBoost	3	0.968	0.977	0.422	0.000	0.436
PhiUSIIL	minus artifact cols	LR	3	0.917	0.888	0.321	0.000	0.332
PhiUSIIL	minus artifact cols	RF	3	0.958	0.866	0.321	0.000	0.341
PhiUSIIL	minus artifact cols	XGBoost	3	0.965	0.878	0.424	0.000	0.396
Corpus B	lexical_v2	LR	3	0.934	0.865	0.997	0.946	0.395
Corpus B	lexical_v2	RF	3	0.973	0.772	0.421	0.176	0.587
Corpus B	lexical_v2	XGBoost	3	0.980	0.874	0.709	0.160	0.683
Corpus B	minus artifact cols	LR	3	0.936	0.883	0.723	0.152	0.417
Corpus B	minus artifact cols	RF	3	0.970	0.780	0.377	0.149	0.574
Corpus B	minus artifact cols	XGBoost	3	0.972	0.849	0.694	0.188	0.631
Hannousse 2021	lexical_v2	LR	3	0.931	0.868	0.314	0.012	0.186
Hannousse 2021	lexical_v2	RF	3	0.972	0.925	0.565	0.063	0.553
Hannousse 2021	lexical_v2	XGBoost	3	0.979	0.927	0.683	0.076	0.627
Hannousse 2021	minus artifact cols	LR	3	0.930	0.877	0.477	0.021	0.415
Hannousse 2021	minus artifact cols	RF	3	0.969	0.935	0.547	0.043	0.517
Hannousse 2021	minus artifact cols	XGBoost	3	0.968	0.936	0.682	0.041	0.594

Removing the artifact columns has mixed effects in this setting. With PhiUSIIL held out, held-out ROC-AUC *decreases* (XGBoost 0.977 → 0.878), so the columns carried information that transferred. With corpus B held out, logistic regression's over-flagging disappears (FPR 0.95 → 0.15). With Hannousse held out, ROC-AUC rises slightly (XGBoost 0.927 → 0.937) and the realised FPR moves closer to the budget (7.6 % → 4.5 %).

F. Calibration under shift (H2)

In-distribution, all models are well calibrated (ECE 0.002–0.004 on the corpus-A test split), because almost all scores lie near 0 or 1 (Fig. 6). On external corpora, ECE rises to 0.42–0.78. Post-hoc calibration fitted on validation data does not help (Table XI): isotonic regression and Platt scaling leave both the external ECE and the external FPR at the 1 % threshold essentially unchanged (FPR 0.99–1.00 on the four independent corpora). In the leave-one-corpus-out setting, isotonic calibration even *increases* the held-out ECE, for example from 0.132 to 0.150 for XGBoost with PhiUSIIL held out. Calibration repairs probabilities only for the distribution it was fitted on; it cannot compensate for a shift in recording conventions.

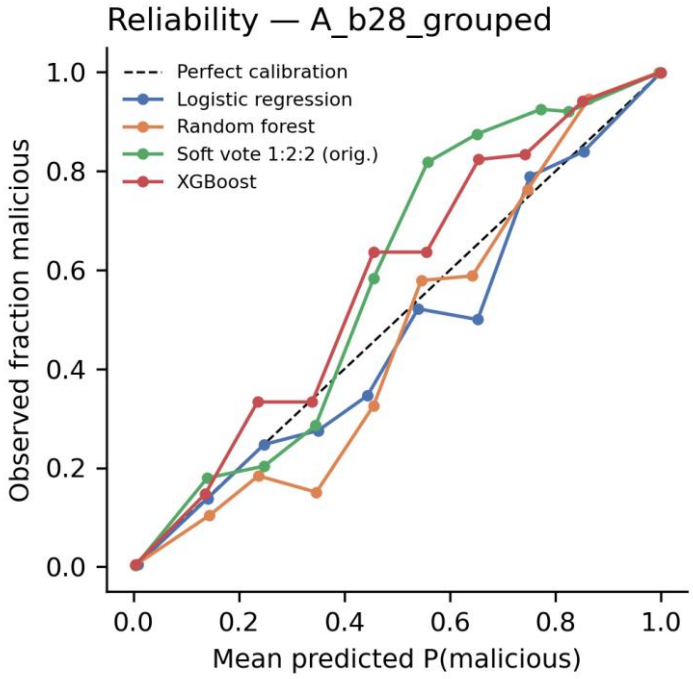


Fig. 6. Reliability diagram on the corpus-A grouped test split (seed 42). Calibration error is small because almost all scores lie near 0 or 1; the few mid-range scores deviate from the diagonal.

TABLE XI. Post-hoc calibration fitted on validation data: expected calibration error (ECE) and FPR at the validation-chosen 1 % threshold, in-distribution and on external corpora (corpus-A models, original features, mean over seeds).

Model	Calibration	ECE test	ECE external (mean of 4)	FPR test	FPR external (mean of 4)	Recall test
LR	none	0.002	0.582	0.010	0.999	0.992
LR	isotonic	0.002	0.582	0.009	0.998	0.991
LR	Platt	0.002	0.582	0.010	0.999	0.992
RF	none	0.004	0.580	0.009	0.999	0.994
RF	isotonic	0.002	0.582	0.006	0.999	0.993
RF	Platt	0.002	0.581	0.009	0.999	0.994
XGBoost	none	0.002	0.582	0.009	0.999	0.995
XGBoost	isotonic	0.002	0.582	0.007	0.999	0.994
XGBoost	Platt	0.002	0.582	0.009	0.999	0.995
Soft vote 1:2:2	none	0.003	0.581	0.009	0.999	0.995
Soft vote 1:2:2	isotonic	0.002	0.582	0.007	0.999	0.994
Soft vote 1:2:2	Platt	0.002	0.582	0.009	0.999	0.995

G. Robustness to label-preserving URL variants (H1)

The sharpest evidence comes from transformations that cannot change a URL's meaning (Fig. 7, Table XII). For the corpus-A models, adding a slash to an empty path (`https://www.example.com` becomes `https://www.example.com/`, which RFC 3986 defines as equivalent) flips the decision for **100 % of benign URLs** (XGBoost; 99.8 % for logistic regression) and lowers ROC-AUC from 0.999 to 0.70. Switching between `http` and `https` flips 100 % of benign and 29 % of malicious decisions, and

removing or adding a leading `www.` flips 84 % of benign decisions for XGBoost. Upper-casing the host, adding a fragment or percent-encoding a path character changes nothing, because the original features ignore them. The model trained on pooled corpora is far more stable: every transformation flips between 3 % and 15 % of its decisions, and none changes its ROC-AUC by more than 0.05.

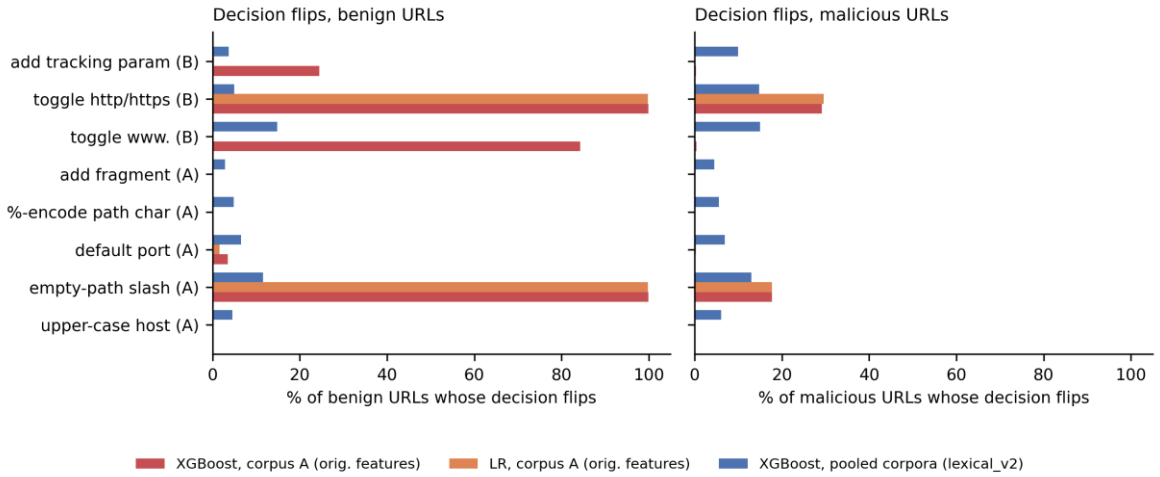


Fig. 7. Share of decisions (threshold 0.5) that flip under label-preserving URL transformations, for benign and malicious URLs (5,000 test URLs). (A) = equivalent under RFC 3986; (B) = presentation change assumed to preserve the label.

TABLE XII. Effect of label-preserving transformations on 5,000 test URLs: share of URLs to which the transformation applies, share of decisions that flip, and ROC-AUC before and after.

Transformation	Tier	Applies to (corpus A)	XGBoost A: flips benign / malicious (%)	LR A: flips (%)	XGBoost A: ROC-AUC	XGBoost pooled: flips (%)	XGBoost pooled: ROC-AUC
Upper-case scheme and host	A	100 %	0.0 / 0.0	0.0 / 0.0	0.999 → 0.999	4.5 / 6.1	0.966 → 0.951
Slash on empty path	A	87 %	100.0 / 17.7	99.8 / 17.7	0.999 → 0.699	11.6 / 13.0	0.966 → 0.959
Explicit default port	A	100 %	3.4 / 0.1	1.6 / 0.3	0.999 → 0.999	6.5 / 6.9	0.966 → 0.947
Percent-encode a path letter	A	13 %	0.0 / 0.0	0.0 / 0.0	0.999 → 0.999	4.9 / 5.5	0.966 → 0.956
Add a fragment	A	100 %	0.2 / 0.0	0.1 / 0.1	0.999 → 1.000	2.8 / 4.5	0.966 → 0.954
Toggle leading www.	B	100 %	84.3 / 0.4	0.2 / 0.2	0.999 → 0.992	14.8 / 15.0	0.966 → 0.918
Toggle http/https	B	100 %	100.0 / 29.1	99.8 / 29.6	0.999 → 0.644	5.0 / 14.8	0.966 → 0.944
Append tracking parameter	B	100 %	24.5 / 0.3	0.4 / 0.2	0.999 → 0.999	3.7 / 10.0	0.966 → 0.951

H. Temporal recall on PhishTank (H3)

PhishTank's verified-online URLs carry submission timestamps, but only 64 of 67,326 predate 2021, so recall can be traced only over 2021–2026 (Fig. 8). The corpus-A models detect 98.8–99.8 % of PhishTank URLs in every year, which, given their external false-positive rates, reflects over-flagging rather than detection skill. The LOCO models show no consistent trend either: XGBoost with Hannousse held out ranges between 0.51 and 0.72 without a monotonic pattern, although the share of https phishing rose from 91 % in 2021 to 98 % in 2024. H3 is therefore not supported over this period.

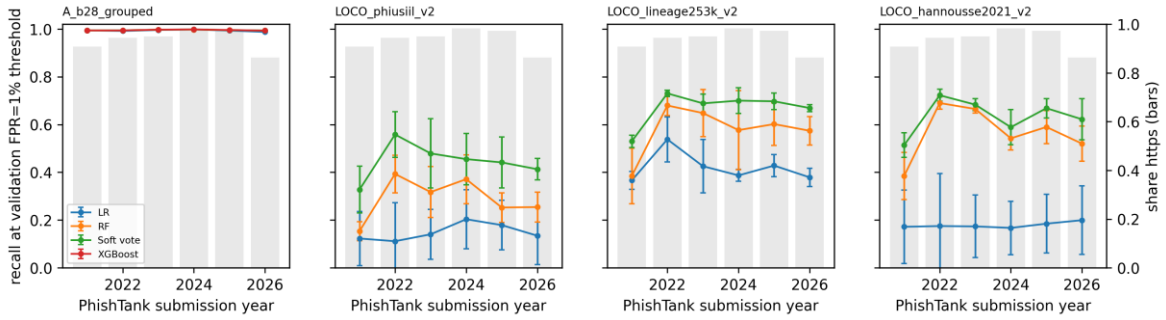


Fig. 8. Recall on PhishTank verified phishing URLs by submission year at the validation-chosen 1 % FPR threshold (lines) and share of https URLs per year (bars). Years with fewer than 100 URLs omitted.

I. Computational cost

All models are cheap enough for inline use (Table XIII). Feature extraction costs about 0.08 ms per URL for the original features and about 0.9 ms for lexical_v2 on the test laptop. Batch scoring takes 0.005 ms per URL for logistic regression, 0.008 ms for XGBoost and 0.02–0.03 ms for random forest and the soft vote. Model size differs by four orders of magnitude: 16 kB for logistic regression, 0.37 MB for XGBoost, 55 MB for random forest and 56 MB for the soft vote, whose size and fit time (45 s versus 10 s for XGBoost) bring no accuracy benefit (Section VIII-B).

TABLE XIII. Computational cost on the test laptop (Intel i7-1355U, CPU only), original features, corpus A, median over seeds. Latency excludes feature extraction.

Model	Training time (s, 140k URLs)	Scoring latency (ms / URL, batch)	Model size
LR	2.1	0.0049	16 kB
XGBoost	9.9	0.0078	366 kB
RF	34.0	0.0223	55.4 MB
Soft vote 1:2:2	44.6	0.0315	55.8 MB

IX. Discussion

What the high scores measure. The baseline reaches a ROC-AUC of 0.999 whether or not registered domains are shared between training and test data, its decisions flip when a slash is added to a homepage URL, and it flags essentially all benign URLs of four independently collected corpora. The simplest explanation consistent with all three observations is that the model learns *how each class was recorded* — benign URLs as `https://www.` homepages and phishing URLs as everything else — rather than properties of phishing. This is the shortcut-learning pitfall described by Arp et al. [12], instantiated here with named and measurable artifacts. It agrees with the benign-homepage effect observed for login pages [11], with the Alexa index-page bias noted by Sabir et al. [16] and with the `https:///www.` shortcut reported in a recent preprint [75]. The original system's verdict on `https://google.com/` is the same phenomenon seen through a single URL.

Why cross-corpus results vary. Because the artifacts change direction between corpora, a model's external performance depends largely on whether the target corpus shares the training corpus's recording conventions. This explains the below-chance ROC-AUC on ISCX-2016 and the Kaggle corpus, and why published cross-dataset results range from near chance [9, 74] to moderate [41]: the outcome is mostly a property of the dataset pair, not of the model.

Model choice is secondary; data choice is not. In-distribution, logistic regression, random forest, XGBoost and the soft vote differ by at most 0.001 ROC-AUC, and the ensemble is statistically indistinguishable from XGBoost. Externally the same models differ by up to 0.2, in an order unrelated to their in-distribution ranking. Training data drawn from several corpora had a far larger effect than any modelling choice: it raised the held-out ROC-AUC of XGBoost from 0.35–0.75 (single-corpus training,

external corpora) to 0.87–0.98 (0.77–0.98 across all three models) and reduced flip rates under label-preserving transformations from up to 100 % to at most 15 %.

Feature removal is not enough. Dropping artifact columns neither stops a probe from identifying the source corpus nor repairs external behaviour, because correlated features carry the same information. Canonicalising the URL string before feature extraction removes the scheme and `www.` information by construction and sharply reduces corpus identifiability; what it cannot remove is a sampling artifact such as a homepage-only benign class, which must be fixed when the data are collected.

Thresholds are the operational failure. Even where ranking transfers, a threshold chosen for a 1 % false-positive budget on one distribution over-flags on one unseen corpus and under-flags on another, and post-hoc calibration does not change this. Deployment therefore needs threshold recalibration on traffic from the target environment, a step the public-benchmark protocol never exercises.

Answers to the research questions. *RQ1:* in-distribution performance on PhiUSIIL is carried almost entirely by the scheme and homepage artifacts; domain memorisation adds nothing measurable there. *RQ2:* the artifacts change strength and direction across corpora, which explains why external performance ranges from moderate to below chance. *RQ3:* training on diverse corpora restores ranking (held-out ROC-AUC 0.87–0.98 for XGBoost and 0.77–0.98 across models) and stability, but neither feature removal nor calibration makes operating thresholds transferable. H1 and H2 are supported; H3 is not supported over 2021–2026.

Implications for practice. Phishing-URL detectors should be (i) trained on several independently collected corpora; (ii) evaluated on held-out *corpora* rather than held-out rows, at fixed false-positive budgets, with precision reported at realistic prevalence; (iii) fed canonicalised URLs so that recording format cannot act as a feature; (iv) checked with a dataset-identification probe and label-preserving transformations before deployment; and (v) recalibrated on traffic from the deployment environment. A deployed system should present its output as a risk estimate, never as a guarantee that a URL is safe.

X. Threats to Validity

Construct validity. All corpora are public compilations with their own artifacts, so "external" means "differently collected", not "real user traffic"; our results expose failure modes rather than estimate deployment performance. Labels are inherited from blacklists and curated lists and were not re-verified. Tier B transformations (scheme and `www.` toggles) are assumed, not proven, to preserve the label.

Internal validity. The original project's dataset could not be obtained; conclusions about it rest on PhiUSIIL, whose benign class has the same `https://www.` form, and on a size-matched sample of its probable source lineage. Registered-domain grouping does not remove near-duplicate phishing pages that share a kit across different domains. The canonicalised representation and the leave-one-corpus-out design were added after the dataset probe and after inspecting the first baseline run, and are reported as such; no hyper-parameter or threshold was tuned on test or external data.

External validity. All labelled corpora are roughly balanced, so precision at realistic prevalence is derived analytically. Timestamped data are positive-only and concentrated in 2021–2026. Only linear and tree-based models on lexical features were evaluated; character-level and transformer models, which see the raw scheme and `www.` tokens directly, may behave differently.

Conclusion validity. Results are averaged over two to five seeds per run (Table IV), and paired comparisons use bootstrap intervals and McNemar's test; differences below about 0.001 ROC-AUC should not be interpreted.

XI. Conclusion and Future Work

Lexical phishing-URL detectors evaluated on a public corpus can reach a ROC-AUC of 0.999 while flagging 99–100 % of the benign URLs of four other public corpora, and while flipping their decision when a slash is added to a homepage URL. We traced this to how each corpus recorded its classes — the scheme, the `www.` prefix and homepage-only benign URLs — whose predictive direction reverses between corpora, so that the source corpus of a URL can be predicted from lexical features with up to 83 % accuracy against 17 % chance. Removing artifact features and calibrating probabilities do not fix the problem. Training on several corpora restores much of the ranking ability and stability on unseen corpora but leaves operating thresholds unreliable. For the system we started from, the weighted ensemble adds cost without accuracy, and a parsing defect disables its host features for scheme-less URLs.

Future work will evaluate character-level CNNs on raw and canonicalised URLs and canonicalised features in the leave-one-corpus-out setting, build a benign URL collection sampled from real browsing traffic with paths and realistic prevalence, and study threshold adaptation with a small amount of labelled data from the target environment.

References

[1] A. Oest et al., “Sunrise to Sunset: Analyzing the End-to-end Life Cycle and Effectiveness of Phishing Attacks at Scale,” in 29th USENIX Security Symposium (USENIX Security 20), 2020. [Online]. Available: https://www.usenix.org/conference/usenixsecurity20/presentation/oest-sunrise.

[2] A. Oest et al., “PhishTime: Continuous Longitudinal Measurement of the Effectiveness of Anti-phishing Blacklists,” in 29th USENIX Security Symposium (USENIX Security 20), pp. 379–396, 2020. [Online]. Available: https://www.usenix.org/conference/usenixsecurity20/presentation/oest-phishtime.

[3] J. Ma, L. K. Saul, S. Savage, and G. M. Voelker, “Beyond blacklists: learning to detect malicious web sites from suspicious URLs,” in Proceedings of the 15th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD '09), pp. 1245-1254, 2009, doi: 10.1145/1557019.1557153.

[4] A. Le, A. Markopoulou, and M. Faloutsos, “PhishDef: URL names say it all,” in 2011 Proceedings IEEE INFOCOM, pp. 191-195, doi: 10.1109/INFCOM.2011.5934995.

[5] M. S. I. Mamun, M. A. Rathore, A. H. Lashkari, N. Stakhanova, and A. A. Ghorbani, “Detecting Malicious URLs Using Lexical Analysis,” in Network and System Security (NSS 2016), LNCS 9955, pp. 467-482, Springer, doi: 10.1007/978-3-319-46298-1_30.

[6] O. K. Sahingoz, E. Buber, O. Demir, and B. Diri, “Machine learning based phishing detection from URLs,” Expert Systems with Applications, 2019, doi: 10.1016/j.eswa.2018.09.029.

[7] S. Sankaranarayanan, A. T. Sivachandran, A. S. Mohd Khairuddin, K. Hasikin, and A. R. Wahab Sait, “An ensemble classification method based on machine learning models for malicious Uniform Resource Locators (URL),” PLOS ONE, 2024, doi: 10.1371/journal.pone.0302196.

[8] H. A. Khan et al., “Explainable AI for phishing URL detection: a Bayesian-optimized stacking ensemble framework with SHAP-guided feature learning,” Frontiers in Artificial Intelligence, 2026, doi: 10.3389/frai.2026.1854934.

[9] M. Mia, D. Derakhshan, and M. M. A. Pritom, “Can Features for Phishing URL Detection Be Trusted Across Diverse Datasets? A Case Study with Explainable AI,” in Proceedings of the 11th International Conference on Networking, Systems, and Security (NSysS 2024), pp. 137–145, doi: 10.1145/3704522.3704532.

[10] F. Rashid, B. Doyle, S. C. Han, and S. Seneviratne, “Phishing URL detection generalisation using Unsupervised Domain Adaptation,” Computer Networks, 2024, doi: 10.1016/j.comnet.2024.110398.

[11] M. Sanchez-Paniagua, E. Fidalgo Fernandez, E. Alegre, W. Al-Nabki, and V. Gonzalez-Castro, “Phishing URL Detection: A Real-Case Scenario Through Login URLs,” IEEE Access, 2022, doi: 10.1109/ACCESS.2022.3168681.

[12] D. Arp et al., “Dos and Don'ts of Machine Learning in Computer Security,” in 31st USENIX Security Symposium (USENIX Security 22), pp. 3971–3988, arXiv:2010.09470, 2022. [Online]. Available: https://www.usenix.org/conference/usenixsecurity22/presentation/arp.

[13] S. Marchal and N. Asokan, “On Designing and Evaluating Phishing Webpage Detection Techniques for the Real World,” in 11th USENIX Workshop on Cyber Security Experimentation and Test (CSET 18), 2018. [Online]. Available: https://www.usenix.org/conference/cset18/presentation/marchal.

[14] A. E. Aassal, S. Baki, A. Das, and R. M. Verma, “An In-Depth Benchmarking and Evaluation of Phishing Detection Research for Security Needs,” IEEE Access, pp. 22170–22192, 2020, doi: 10.1109/ACCESS.2020.2969780.

[15] A. Hannousse and S. Yahiouche, “Towards benchmark datasets for machine learning based website phishing detection: An experimental study,” Engineering Applications of Artificial Intelligence, 2021, doi: 10.1016/j.engappai.2021.104347.

[16] B. Sabir, M. A. Babar, R. Gaire, and A. Abuadbba, “Reliability and Robustness analysis of Machine Learning based Phishing URL Detectors,” IEEE Transactions on Dependable and Secure Computing, 2022, doi: 10.1109/TDSC.2022.3218043.

[17] D. K. McGrath and M. Gupta, “Behind Phishing: An Examination of Phisher Modi Operandi,” in USENIX Workshop on Large-Scale Exploits and Emergent Threats (LEET '08), 2008. [Online]. Available: https://www.usenix.org/legacy/event/leet08/tech/full_papers/mcgrath/mcgrath.pdf.

[18] S. Garera, N. Provos, M. Chew, and A. D. Rubin, “A framework for detection and measurement of phishing attacks,” in Proceedings of the 2007 ACM Workshop on Recurring Malcode (WORM '07), pp. 1-8, doi: 10.1145/1314389.1314391.

[19] J. Ma, L. K. Saul, S. Savage, and G. M. Voelker, “Identifying suspicious URLs: an application of large-scale online learning,” in Proceedings of the 26th International Conference on Machine Learning (ICML '09), pp. 681-688, 2009, doi: 10.1145/1553374.1553462.

[20] J. Ma, L. K. Saul, S. Savage, and G. M. Voelker, “Learning to detect malicious URLs,” ACM Transactions on Intelligent Systems and Technology, 2011, doi: 10.1145/1961189.1961202.

[21] A. Blum, B. Wardman, T. Solorio, and G. Warner, “Lexical feature based phishing URL detection using online learning,” in Proceedings of the 3rd ACM Workshop on Artificial Intelligence and Security (AISec '10), pp. 54-60, 2010, doi: 10.1145/1866423.1866434.

[22] R. Verma and K. Dyer, “On the Character of Phishing URLs: Accurate and Robust Statistical Learning Classifiers,” in Proceedings of the 5th ACM Conference on Data and Application Security and Privacy (CODASPY '15), pp. 111-122, 2015, doi: 10.1145/2699026.2699115.

[23] S. Marchal, J. Francois, R. State, and T. Engel, “PhishStorm: Detecting Phishing With Streaming Analytics,” IEEE Transactions on Network and Service Management, 2014, doi: 10.1109/TNSM.2014.2377295.

[24] R. Verma and A. Das, “What's in a URL: Fast Feature Extraction and Malicious URL Detection,” in Proceedings of the 3rd ACM International Workshop on Security and Privacy Analytics (IWSPA '17), pp. 55-63, 2017, doi: 10.1145/3041008.3041016.

[25] A. Prasad and S. Chandra, “PhiUSIIL: A diverse security profile empowered phishing URL detection framework based on similarity index and incremental learning,” Computers & Security, 2024, doi: 10.1016/j.cose.2023.103545.

[26] R. S. Rao, T. Vaishnavi, and A. R. Pais, “CatchPhish: detection of phishing websites by inspecting URLs,” Journal of Ambient Intelligence and Humanized Computing, 2020, doi: 10.1007/s12652-019-01311-4.

[27] K. L. Chiew, C. L. Tan, K. Wong, K. S. C. Yong, and W. K. Tiong, “A new hybrid ensemble feature selection framework for machine learning-based phishing detection system,” Information Sciences, 2019, doi: 10.1016/j.ins.2019.01.064.

[28] M. Sameen, K. Han, and S. O. Hwang, “PhishHaven-An Efficient Real-Time AI Phishing URLs Detection System,” IEEE Access, 2020, doi: 10.1109/ACCESS.2020.2991403.

[29] J. Lee, P. Ye, R. Liu, D. M. Divakaran, and M. C. Chan, “Building Robust Phishing Detection System: an Empirical Analysis,” in NDSS Workshop on Measurements, Attacks, and Defenses for the Web (MADWeb 2020). [Online]. Available: https://www.ndss-symposium.org/wp-content/uploads/2020/02/23007-paper.pdf.

[30] J. Saxe and K. Berlin, “eXpose: A Character-Level Convolutional Neural Network with Embeddings For Detecting Malicious URLs, File Paths and Registry Keys,” arXiv preprint arXiv:1702.08568, 2017. [Online]. Available: https://arxiv.org/abs/1702.08568.

[31] A. C. Bahnsen, E. C. Bohorquez, S. Villegas, J. Vargas, and F. A. Gonzalez, “Classifying phishing URLs using recurrent neural networks,” in 2017 APWG Symposium on Electronic Crime Research (eCrime), doi: 10.1109/ecrime.2017.7945048.

[32] H. Le, Q. Pham, D. Sahoo, and S. C. H. Hoi, “URLNet: Learning a URL Representation with Deep Learning for Malicious URL Detection,” arXiv preprint arXiv:1802.03162, 2018. [Online]. Available: https://arxiv.org/abs/1802.03162.

[33] Y. Huang, Q. Yang, J. Qin, and W. Wen, “Phishing URL Detection via CNN and Attention-Based Hierarchical RNN,” in 2019 18th IEEE International Conference on Trust, Security and Privacy in Computing and Communications / 13th IEEE International Conference on Big Data Science and Engineering (TrustCom/BigDataSE), pp. 112-119, doi: 10.1109/TrustCom/BigDataSE.2019.00024.

[34] W. Wang, F. Zhang, X. Luo, and S. Zhang, “PDRCNN: Precise Phishing Detection with Recurrent Convolutional Neural Networks,” Security and Communication Networks, 2019, doi: 10.1155/2019/2595794.

[35] F. Tajaddodianfar, J. W. Stokes, and A. Gururajan, “Texception: A Character/Word-Level Deep Learning Model for Phishing URL Detection,” in ICASSP 2020 - IEEE International Conference on Acoustics, Speech and Signal Processing, pp. 2857-2861, doi: 10.1109/ICASSP40776.2020.9053670.

[36] A. S. Bozkir, F. C. Dalgic, and M. Aydos, “GramBeddings: A New Neural Network for URL Based Identification of Phishing Web Pages Through N-gram Embeddings,” Computers & Security, 2023, doi: 10.1016/j.cose.2022.102964.

[37] O. K. Sahingoz, E. Buber, and E. Kugu, “DEPHIDES: Deep Learning Based Phishing Detection System,” IEEE Access, 2024, doi: 10.1109/ACCESS.2024.3352629.

[38] P. Maneriker, J. W. Stokes, E. G. Lazo, D. Carutasu, F. Tajaddodianfar, and A. Gururajan, “URLTran: Improving Phishing URL Detection Using Transformers,” in MILCOM 2021 - IEEE Military Communications Conference, pp. 197-204, doi: 10.1109/MILCOM52596.2021.9653028.

[39] Y. Wang, W. Zhu, H. Xu, Z. Qin, K. Ren, and W. Ma, “A Large-Scale Pretrained Deep Model for Phishing URL Detection,” in ICASSP 2023 - IEEE International Conference on Acoustics, Speech and Signal Processing, pp. 1-5, doi: 10.1109/ICASSP49357.2023.10095719.

[40] Y. Li, Y. Liu, P. Li, Y. Jia, and Y. Wang, “Continuous Multi-Task Pre-training for Malicious URL Detection and Webpage Classification (URLBERT),” Computer Networks, 2025, doi: 10.1016/j.comnet.2025.111513.

[41] R. Liu et al., “TransURL: Improving malicious URL detection with multi-layer Transformer encoding and multi-scale pyramid features,” Computer Networks, 2024, doi: 10.1016/j.comnet.2024.110707.

[42] K. Haynes, H. Shirazi, and I. Ray, “Lightweight URL-based phishing detection using natural language processing transformers for mobile devices,” in Procedia Computer Science, 191, pp. 127-134 (FNC/MobiSPC 2021), doi: 10.1016/j.procs.2021.07.040.

[43] S. S. Roy and S. Nilizadeh, “PhishLang: A Real-Time, Fully Client-Side Phishing Detection Framework Using MobileBERT,” arXiv:2408.05667 (v3, Apr 2025); CC BY 4.0, 2024. [Online]. Available: https://arxiv.org/abs/2408.05667.

[44] F. Trad and A. Chehab, “Prompt Engineering or Fine-Tuning? A Case Study on Phishing Detection with Large Language Models,” Machine Learning and Knowledge Extraction, 2024, doi: 10.3390/make6010018.

[45] H. Kibriya, R. Amin, S. S. Alshamrani, S. Rehman, M. Hassan, and F. S. Alsubaei, “Lightweight malicious URL detection using deep learning and large language models,” Scientific Reports, 2025, doi: 10.1038/s41598-025-26653-2.

[46] A. Aljofey, Q. Jiang, Q. Qu, M. Huang, and J.-P. Niyigena, “An Effective Phishing Detection Model Based on Character Level Convolutional Neural Network from URL,” Electronics, 2020, doi: 10.3390/electronics9091514.

[47] S. M. Lundberg and S.-I. Lee, “A Unified Approach to Interpreting Model Predictions,” in Advances in Neural Information Processing Systems 30 (NeurIPS 2017). [Online]. Available: https://arxiv.org/abs/1705.07874.

[48] S. M. Lundberg et al., “From local explanations to global understanding with explainable AI for trees,” Nature Machine Intelligence 2(1):56-67, 2020, doi: 10.1038/s42256-019-0138-9.

[49] M. Al-Fayoumi, B. Alhijawi, Q. A. Al-Haija, and R. Armoush, “XAI-PhD: Fortifying Trust of Phishing URL Detection Empowered by Shapley Additive Explanations,” International Journal of Online and Biomedical Engineering (iJOE) 20(11):80-101, 2024, doi: 10.3991/ijoe.v20i11.49533.

[50] T. Kehkashan et al., “Explainable phishing website detection for secure and sustainable cyber infrastructure,” Scientific Reports 15, article 41751, 2025, doi: 10.1038/s41598-025-27984-w.

[51] G. Apruzzese, M. Conti, and Y. Yuan, “SpacePhish: The Evasion-space of Adversarial Attacks against Phishing Website Detectors using Machine Learning,” in Proceedings of the 38th Annual Computer Security Applications Conference (ACSAC 2022), pp. 171–185, doi: 10.1145/3564625.3567980.

[52] A. AlEroud and G. Karabatis, “Bypassing Detection of URL-based Phishing Attacks Using Generative Adversarial Deep Neural Networks,” in Proceedings of the Sixth International Workshop on Security and Privacy Analytics (IWSPA '20), pp. 53-60, 2020, doi: 10.1145/3375708.3380315.

[53] A. C. Bahnsen, I. Torroledo, L. D. Camacho, and S. Villegas, “DeepPhish: Simulating Malicious AI,” Black Hat Europe 2018 (talk, 5 Dec 2018, slides); also presented at DEF CON 26 AI Village. [Online]. Available: https://i.blackhat.com/eu-18/Wed-Dec-5/eu-18-CorreaBahnsen-DeepPhish-Simulating-Malicious-AI.pdf.

[54] G. Apruzzese, H. S. Anderson, S. Dambra, D. Freeman, F. Pierazzi, and K. A. Roundy, “"Real Attackers Don't Compute Gradients": Bridging the Gap Between Adversarial ML Research and Practice,” in 2023 IEEE Conference on Secure and Trustworthy Machine Learning (SaTML), doi: 10.1109/SaTML54575.2023.00031.

[55] H. Hu, S. T. K. Jan, Y. Wang, and G. Wang, “Assessing Browser-level Defense against IDN-based Phishing,” in 30th USENIX Security Symposium, pp. 3739-3756, 2021. [Online]. Available: https://www.usenix.org/conference/usenixsecurity21/presentation/hu-hang.

[56] K. Tian, S. T. K. Jan, H. Hu, D. Yao, and G. Wang, “Needle in a Haystack: Tracking Down Elite Phishing Domains in the Wild,” in Proceedings of the Internet Measurement Conference 2018 (IMC '18), Boston, MA, doi: 10.1145/3278532.3278569.

[57] P. Kintis et al., “Hiding in Plain Sight: A Longitudinal Study of Combosquatting Abuse,” in Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security (CCS '17), pp. 569-586, doi: 10.1145/3133956.3134002.

[58] A. Niculescu-Mizil and R. Caruana, “Predicting good probabilities with supervised learning,” in Proceedings of the 22nd International Conference on Machine Learning (ICML '05), 2005, doi: 10.1145/1102351.1102430.

[59] C. Guo, G. Pleiss, Y. Sun, and K. Q. Weinberger, “On Calibration of Modern Neural Networks,” in Proceedings of the 34th International Conference on Machine Learning (ICML 2017), PMLR 70. [Online]. Available: https://arxiv.org/abs/1706.04599.

[60] D. Li, T. Qiu, S. Chen, Q. Li, and S. Xu, “Can We Leverage Predictive Uncertainty to Detect Dataset Shift and Adversarial Examples in Android Malware Detection?,” in Annual Computer Security Applications Conference (ACSAC '21), pp. 596-608, 2021, doi: 10.1145/3485832.3485916.

[61] R. Sommer and V. Paxson, “Outside the Closed World: On Using Machine Learning for Network Intrusion Detection,” in 2010 IEEE Symposium on Security and Privacy, doi: 10.1109/SP.2010.25.

[62] S. Axelsson, “The base-rate fallacy and the difficulty of intrusion detection,” ACM Transactions on Information and System Security, 2000, doi: 10.1145/357830.357849.

[63] F. Pendlebury, F. Pierazzi, R. Jordaney, J. Kinder, and L. Cavallaro, “TESSERACT: Eliminating Experimental Bias in Malware Classification across Space and Time,” in 28th USENIX Security Symposium (USENIX Security 19), pp. 729–746, arXiv:1807.07838, 2019. [Online]. Available: https://www.usenix.org/conference/usenixsecurity19/presentation/pendlebury.

[64] Z. Kan et al., “TESSERACT: Eliminating Experimental Bias in Malware Classification across Space and Time (Extended Version),” arXiv preprint, 2024, doi: 10.48550/arXiv.2402.01359.

[65] R. Jordaney et al., “Transcend: Detecting Concept Drift in Malware Classification Models,” in 26th USENIX Security Symposium (USENIX Security 17), pp. 625–642, 2017. [Online]. Available: https://www.usenix.org/conference/usenixsecurity17/technical-sessions/presentation/jordaney.

[66] F. Barbero, F. Pendlebury, F. Pierazzi, and L. Cavallaro, “Transcending TRANSCEND: Revisiting Malware Classification in the Presence of Concept Drift,” in 2022 IEEE Symposium on Security and Privacy (S&P), doi: 10.1109/SP46214.2022.9833659.

[67] Y. Chen, Z. Ding, and D. Wagner, “Continuous Learning for Android Malware Detection,” in 32nd USENIX Security Symposium (USENIX Security 23), pp. 1127–1144, arXiv:2302.04332, 2023. [Online]. Available: https://www.usenix.org/conference/usenixsecurity23/presentation/chen-yizheng.

[68] A. Ejaz, A. N. Mian, and S. Manzoor, “Life-long phishing attack detection using continual learning,” Scientific Reports, 2023, doi: 10.1038/s41598-023-37552-9.

[69] A. Das, S. Baki, A. E. Aassal, R. Verma, and A. Dunbar, “SoK: A Comprehensive Reexamination of Phishing Research From the Security Perspective,” IEEE Communications Surveys & Tutorials, 2020, doi: 10.1109/COMST.2019.2957750.

[70] M. Alam et al., “SoK: PHILTER: Uncovering Security and Functional Gaps in AI-based Phishing Website Detection Literature via an LLM-based Reasoning Framework,” in 35th USENIX Security Symposium (USENIX Security 26), prepublication, 2026. [Online]. Available: https://www.usenix.org/system/files/conference/usenixsecurity26/sec26_prepub_alam.pdf.

[71] V. Le Pochat, T. Van Goethem, S. Tajalizadehkhoob, M. Korczyński, and W. Joosen, “Tranco: A Research-Oriented Top Sites Ranking Hardened Against Manipulation,” in Network and Distributed System Security Symposium (NDSS 2019), doi: 10.14722/ndss.2019.23386.

[72] Q. Scheitle et al., “A Long Way to the Top: Significance, Structure, and Stability of Internet Top Lists,” in Proceedings of the Internet Measurement Conference 2018 (IMC '18), doi: 10.1145/3278532.3278574.

[73] T. Dalton et al., “PhreshPhish: A Real-World, High-Quality, Large-Scale Phishing Website Dataset and Benchmark,” arXiv preprint, 2025. [Online]. Available: https://arxiv.org/abs/2507.10854.

[74] S. Henry, B. U.-A. Stephen, and P. Asuquo, “Ensemble transfer learning for cross-dataset phishing detection,” Franklin Open, 2026, doi: 10.1016/j.fraope.2026.100579.

[75] G. R. Giri, “ShieldPhish: Evaluating Representation Robustness and Cross-Source Generalization in Phishing URL Detection,” Zenodo preprint (submitted to IEEE), 2026, doi: 10.5281/zenodo.23141305.

[76] I. Bhuiyan and T. Bhuiyan, “The Fragility of Phishing Detection Models: Evidence from Cross-Corpus Transfer, Prevalence Shift, Artifact Learning, and Evasion Risk,” Big Data and Cognitive Computing, 2026, doi: 10.3390/bdcc10070211.

[77] Manu Siddhartha (Kaggle user sid321axn), “Malicious URLs dataset (Kaggle: 'Huge dataset of 6,51,191 Malicious URLs'),” Kaggle dataset, 2021. [Online]. Available: https://www.kaggle.com/datasets/sid321axn/malicious-urls-dataset.

[78] Cisco Talos / OpenDNS (community), “PhishTank verified online phishing URL database,” 2026. [Online]. Available: https://phishtank.org/developer_info.php.

[79] abuse.ch / Spamhaus, “URLhaus (abuse.ch) malware URL feed,” 2026. [Online]. Available: https://urlhaus.abuse.ch/api/.

[80] OpenPhish, “OpenPhish Community Feed,” 2026. [Online]. Available: https://openphish.com/phishing_feeds.html.